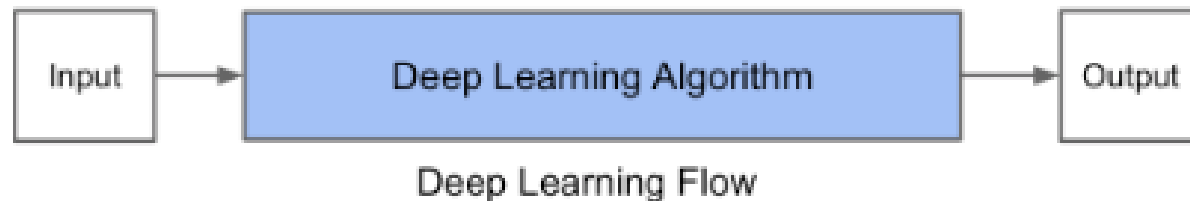


Deep Learning

Dr. Vimal K. Shrivastava
Assistant Professor (II)
School of Electronics Engineering
KIIT Deemed to be University,
Bhubaneswar

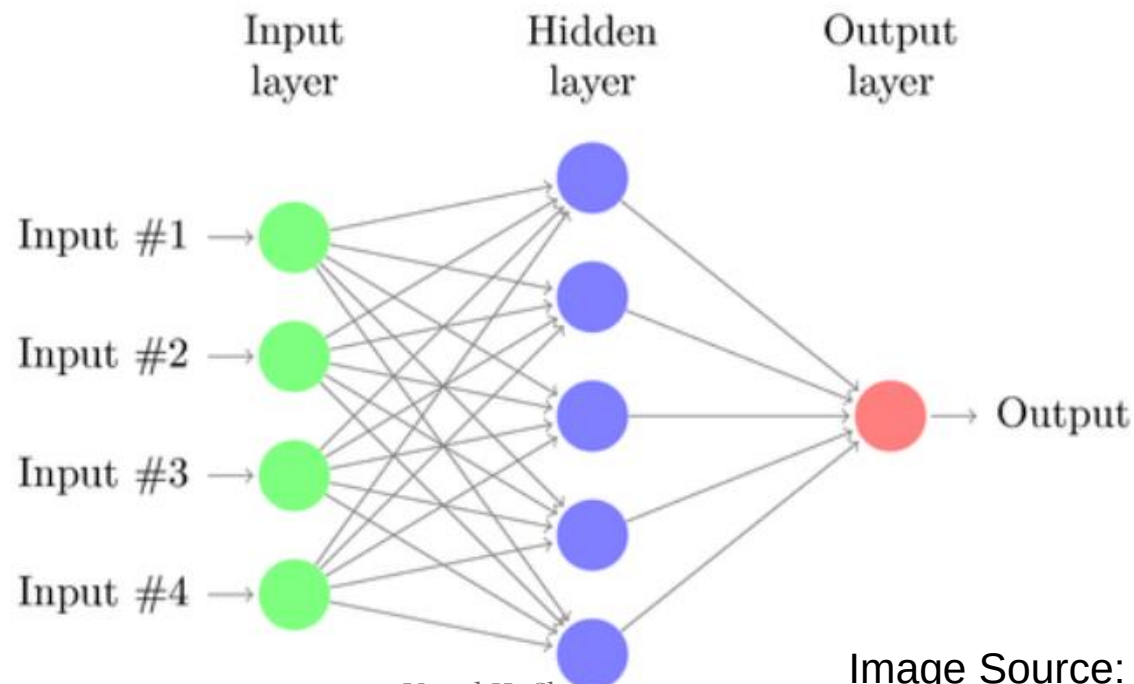
Introduction To Deep Learning

- Deep learning (DL) is a machine learning technique that employs the **deep neural network**.
- In general, it contains more than two hidden layers.
- No need to extract handcrafted features (Raw data can be fed as input to DL).



Limitations of Traditional Neural Networks

- The amount of weights rapidly becomes unmanageable for large images.
- Spatial information is lost when the image is flattened.
- They are not translation invariant.

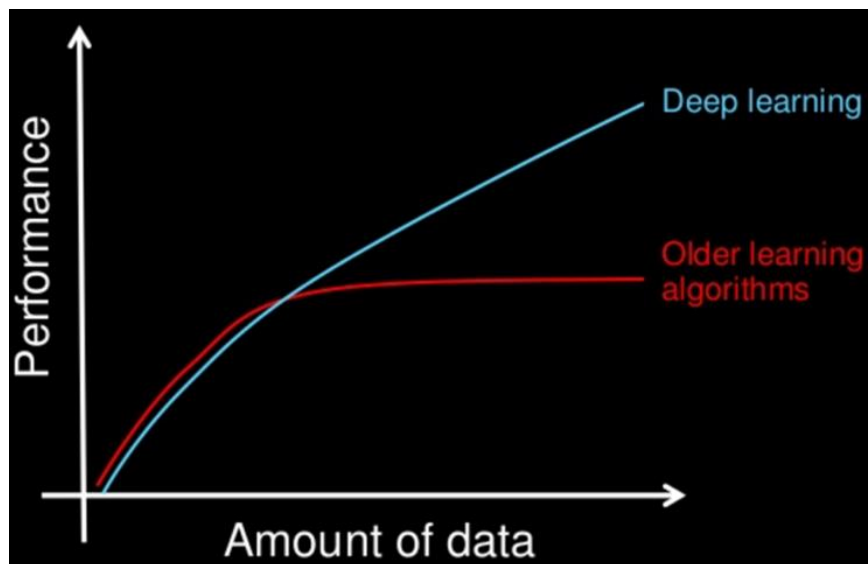


Vimal K. Shrivastava

Image Source: Google Images

Why Deep Learning?

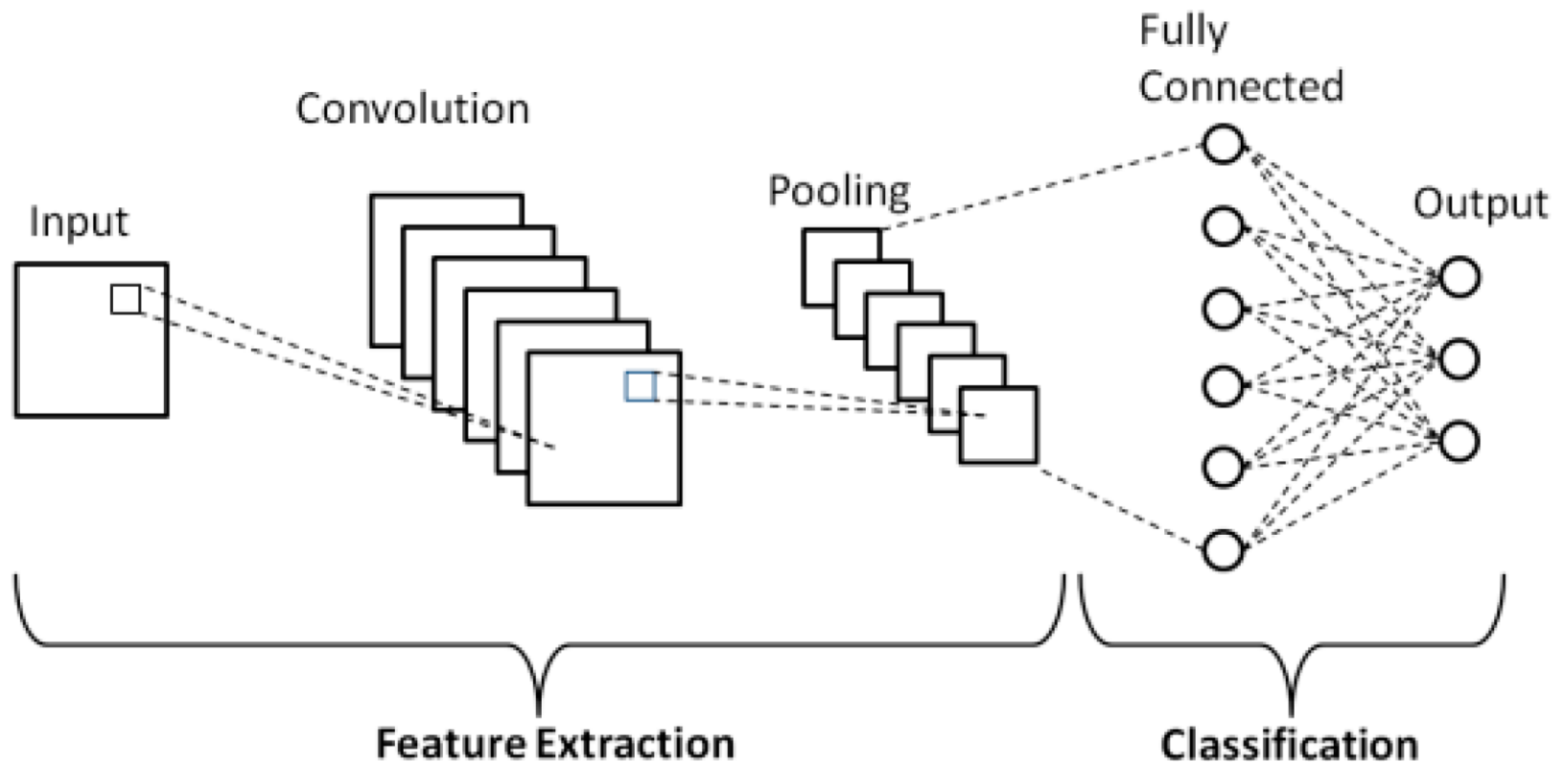
Availability of large datasets



Hardware with high computation power



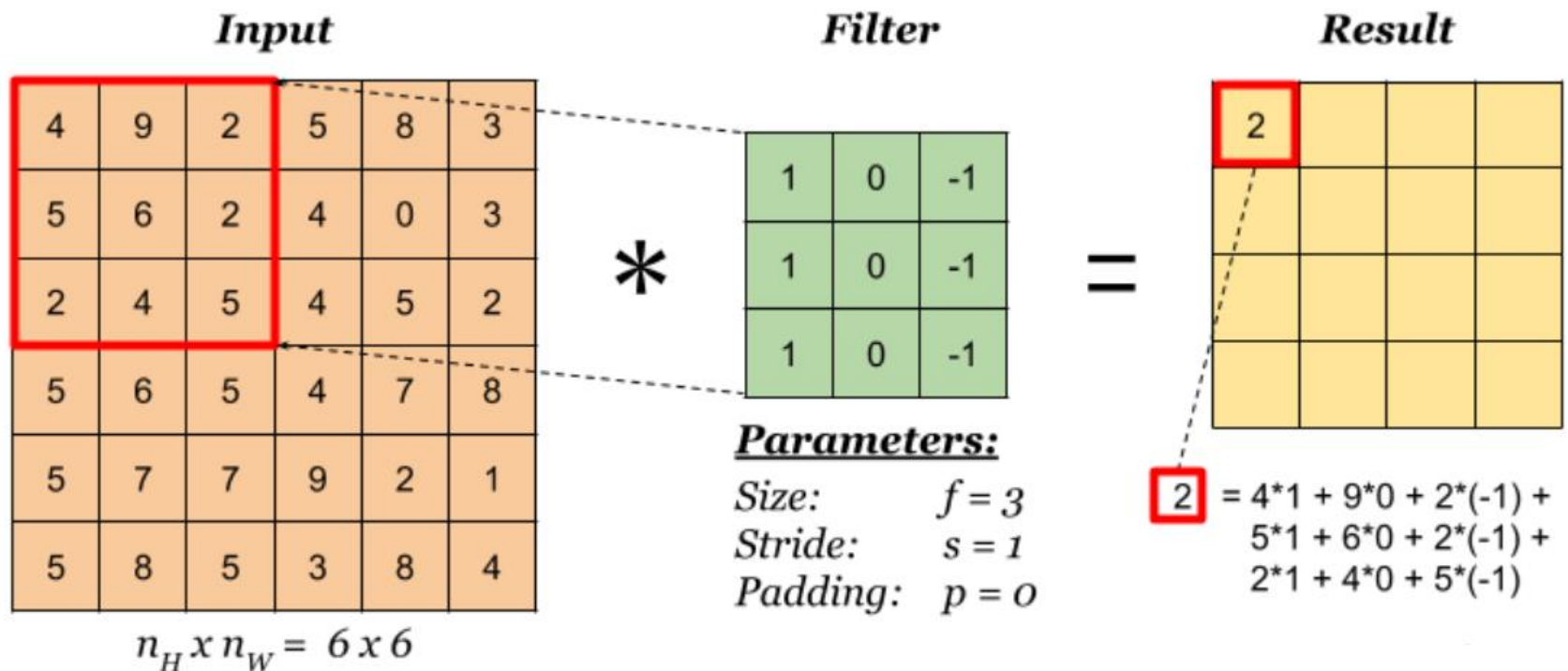
Convolutional Neural Network (CNN)



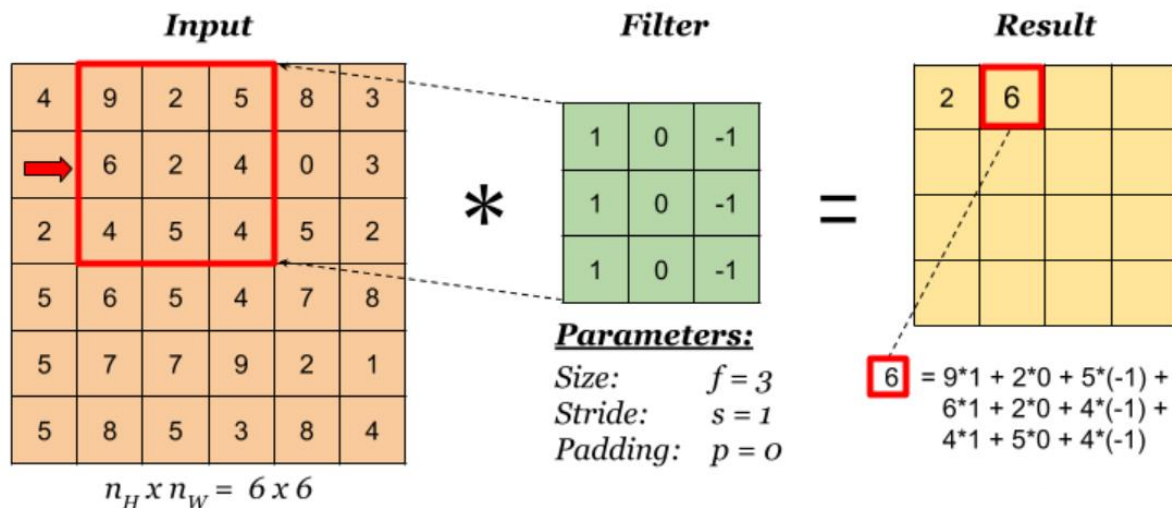
- CNN contains three main types of layers:
 - **Convolutional Layer**
 - **Pooling Layer**
 - **Fully-Connected Layer**

Convolutional Layer

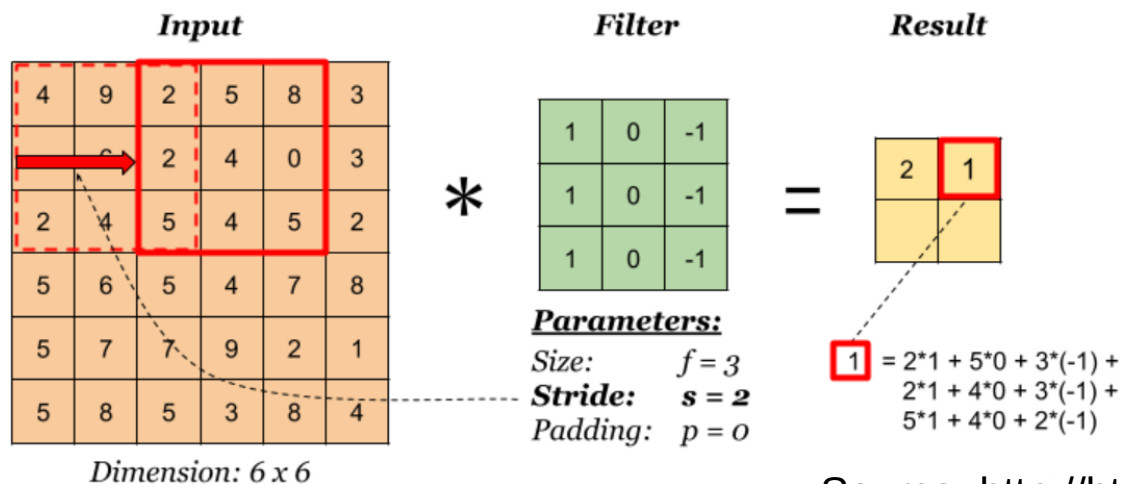
- The Conv layer is the core building block of a CNN.
- **Step 1:** overlay the filter (kernel) to the input, perform element wise multiplication, and add the result.



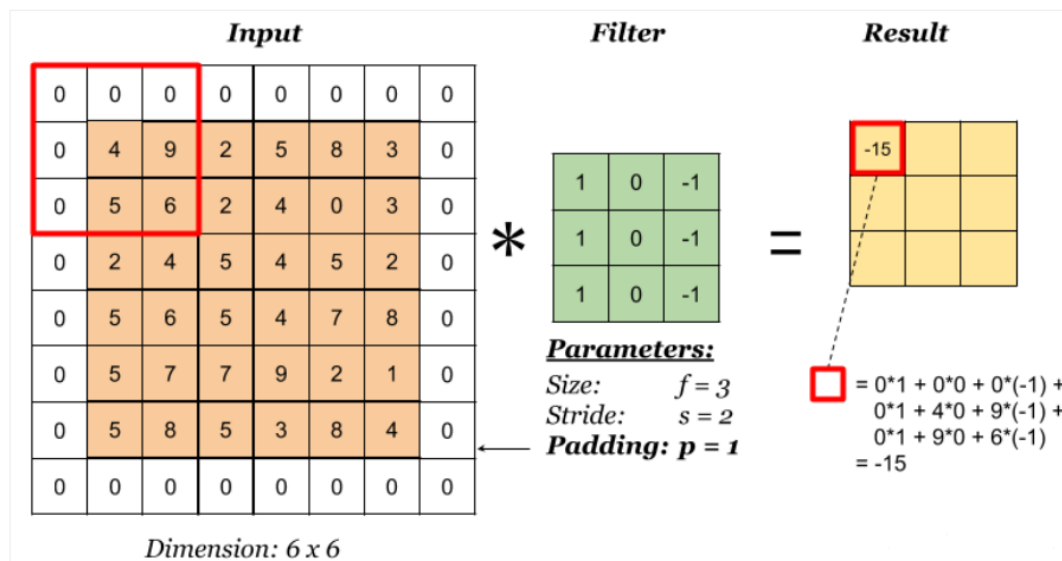
- **Step 2:** move the overlay right one position (stride=1), and do the same calculation to get the next result and so on.



- **Stride** indicates how many cells the filter is moved in the input.

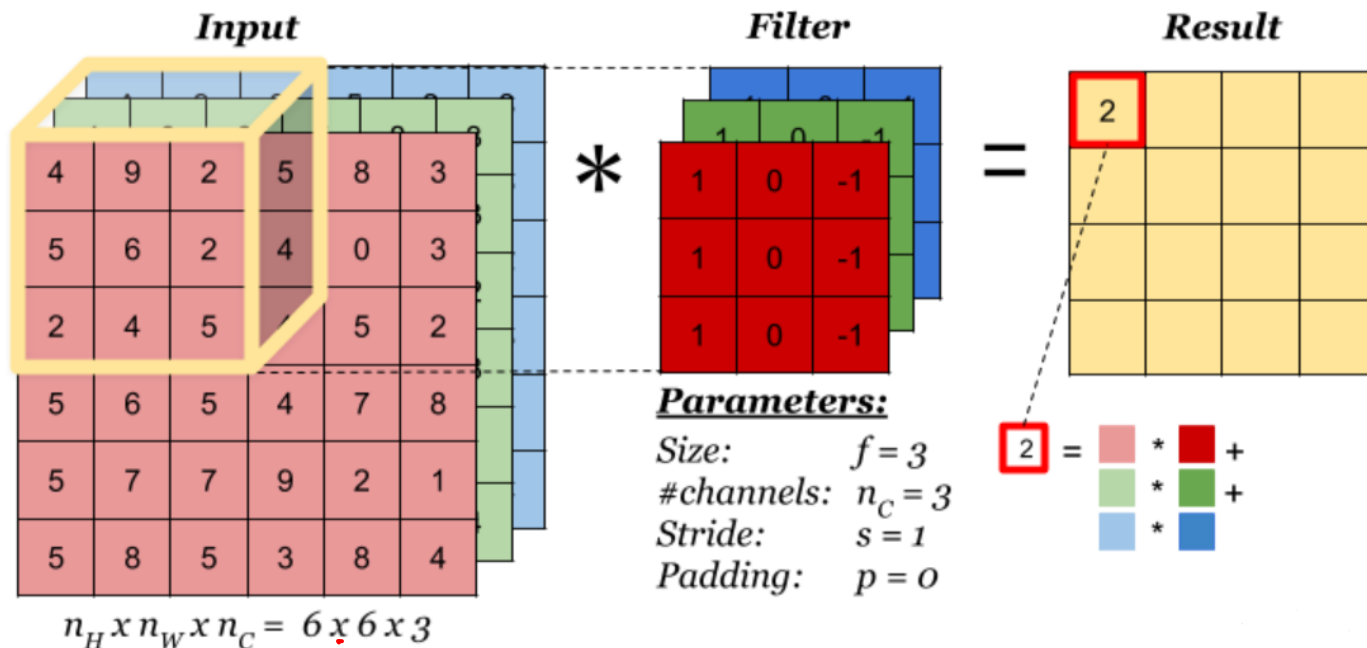


- **Padding:**
- It allows to use a Conv layer without necessarily shrinking the height and width of the feature maps.
- It helps to keep more of the information at the border of an image.

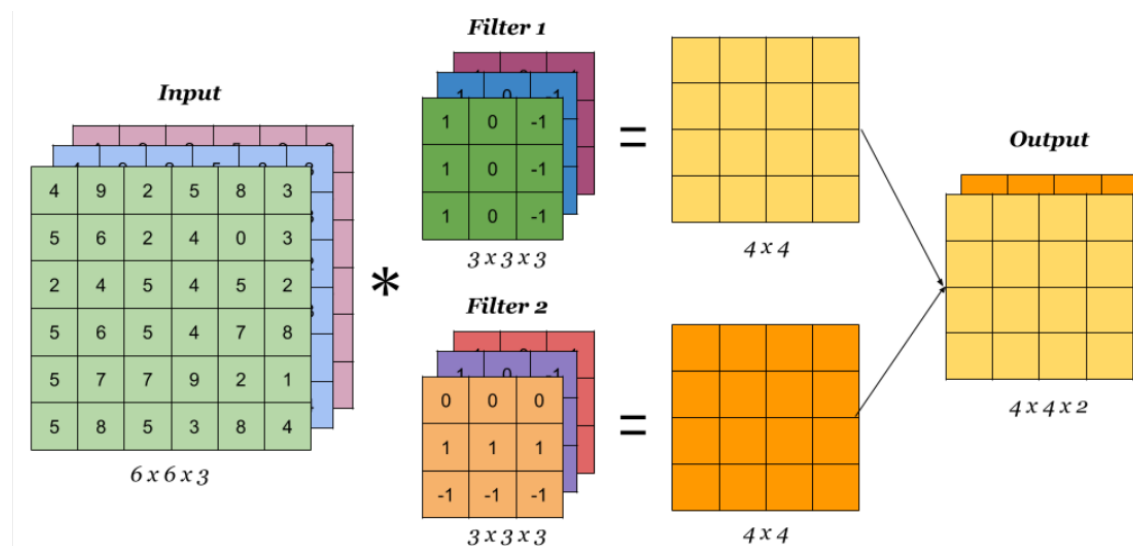


- Some padding terminologies:
 - **Valid:** no padding
 - **Same:** padding so that the output dimension is the same as the input

- **Output Dimension:**
$$n^{[l]} = \left\lfloor \frac{n^{[l-1]} + 2p^{[l-1]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$
- **Convolution Operation on RGB Image:**
- When the input has more than one channels (e.g. RGB image), the filter should have matching number of channels.
- To calculate one output cell, perform convolution on each matching channel, then add the result together.

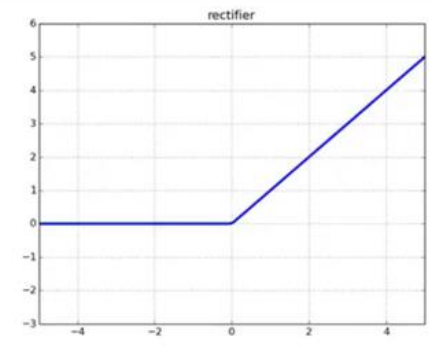


- **Convolution Operation with Multiple Filters:**
- Multiple filters are used in a convolution layer to detect multiple features.
- The output of the layer then will have the same number of channels as the number of filters in the layer.



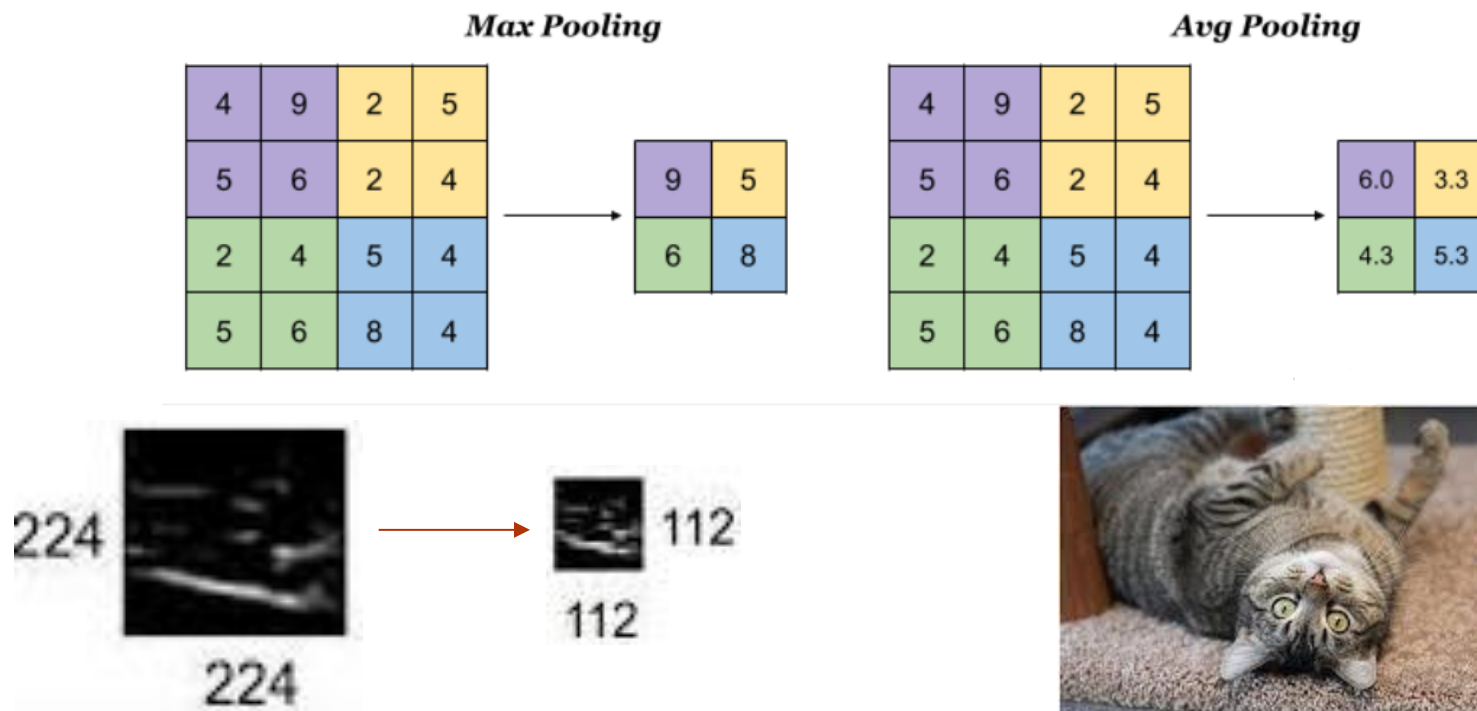
- Finally, an activation function such as ReLU is applied.

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$



Pooling Layer

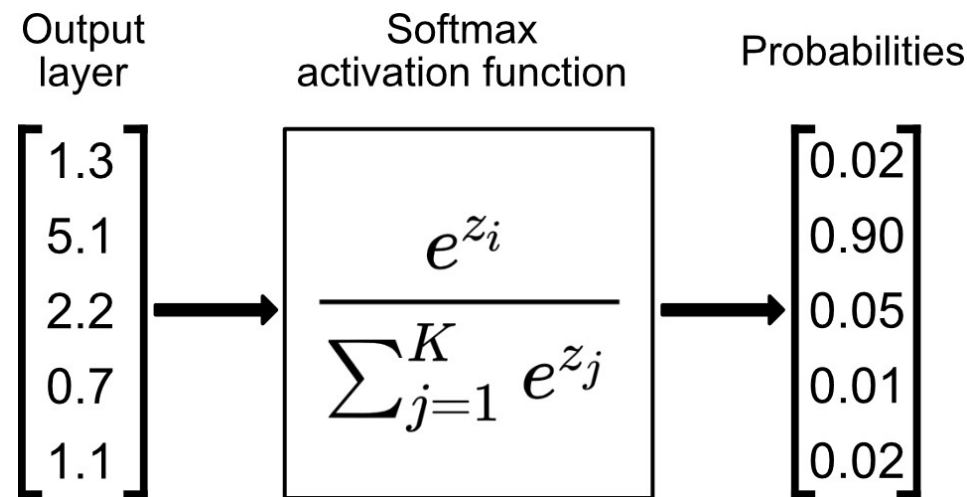
- Pooling layer is used to reduce the size of the feature maps and hence, speed up the calculations.
- Also provides translation invariance property.



- There is no learning parameter in pooling layer.
- When pooling is performed on input with multiple channels, pooling reduces the height and width but keeps number of channels unchanged.

Fully Connected Layer

- **FC layer** takes the output of the previous layer, “flatten” it and turns them into a single vector.
- There may be multiple FC layers in deep CNN.
- The last layer of CNNs has **softmax** function that outputs a probability distribution of classes.



One-hot Encoding

Human-Readable

Pet
Cat
Dog
Turtle
Fish
Cat



Machine-Readable

Cat	Dog	Turtle	Fish
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

- **Problem 1:**
- Suppose we have an input image of size $32 \times 32 \times 3$. This image passes through two layers in a CNN: (1) a Convolutional Layer with 16 filters of 3×3 kernel, stride 1, and no padding, and (2) a Max Pooling Layer with a 2×2 pool size and stride 2. Calculate the size of the feature maps after each layer.
- **Solution:**
- 1. After the Convolutional Layer:

$$\text{Output Height} = \frac{\text{Input Height} - \text{Kernel Height} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

$$\text{Output Width} = \frac{\text{Input Width} - \text{Kernel Width} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

$$\text{Output Height} = \frac{32 - 3 + 2 \times 0}{1} + 1 = 30$$

$$\text{Output Width} = \frac{32 - 3 + 2 \times 0}{1} + 1 = 30$$

Since there are 16 filters, the depth of the feature map will be 16.

So, the output feature map size after the convolutional layer is:

$$30 \times 30 \times 16$$

- 2. After the Max Pooling Layer:

$$\text{Output Height} = \frac{\text{Input Height} - \text{Pool Height}}{\text{Stride}} + 1$$

$$\text{Output Width} = \frac{\text{Input Width} - \text{Pool Width}}{\text{Stride}} + 1$$

$$\text{Output Height} = \frac{30 - 2}{2} + 1 = 15$$

$$\text{Output Width} = \frac{30 - 2}{2} + 1 = 15$$

The depth of the feature map remains the same (16) after pooling.

So, the output feature map size after the max pooling layer is:

$$15 \times 15 \times 16$$

Problem 2:

- Suppose we have an input image of size $64 \times 64 \times 3$. This image is passed through a CNN with two layers: (1) a **Convolutional Layer** with 32 filters of a 5×5 kernel, stride 1, and no padding, and (2) a **Max Pooling Layer** with a 3×3 pool size and stride 2. Calculate the size of the feature maps after each layer.

Convolution vs Fully Connected Layer

- Fully connected layer

- The input image size is $32 \times 32 \times 3$
- Flatten it first (convert to 1-d vector)
- Dimension of input vector: $32 \times 32 \times 3 = 3072 \times 1$
- If there are 10 units (neuron) in the first hidden layer of a FC network, then each unit looks at the full input vector

$$a = f(Z) = f(W^T X + b).$$

Trainable parameters

Weights: 3072×10

Bias: 10

Total trainable parameters :

30730

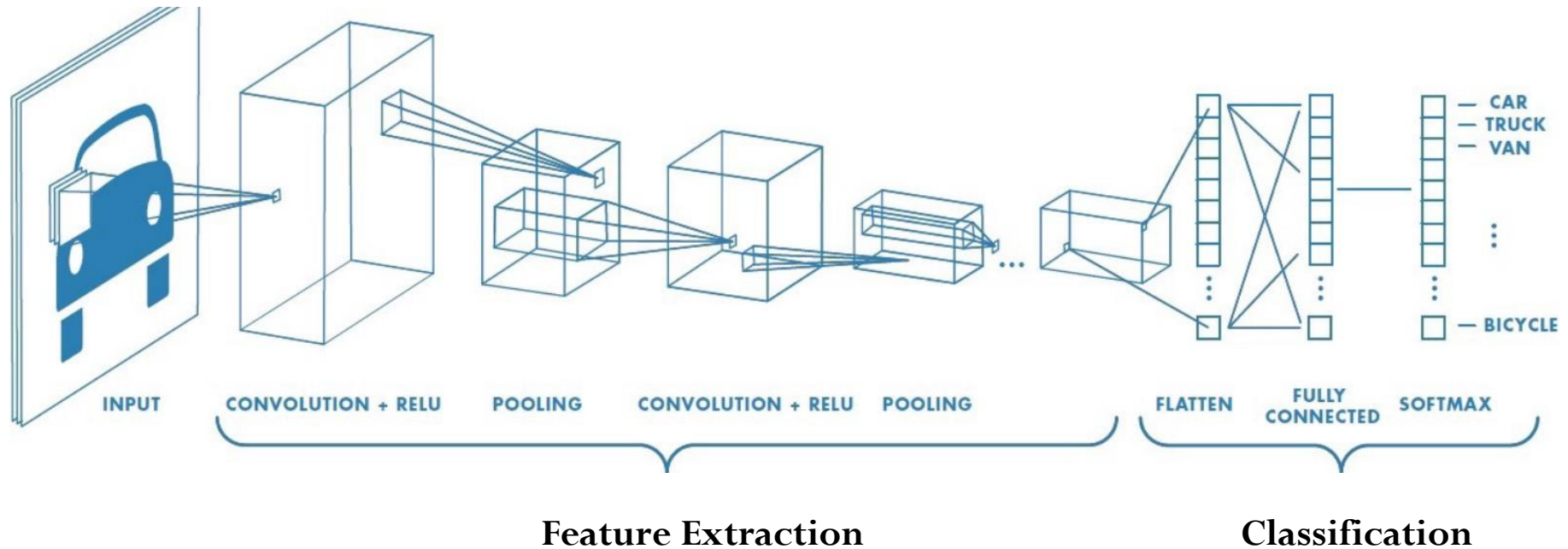
If there are 10 filters, each of size $3 \times 3 \times 3$ in the first Conv layer of the network.

Total trainable parameters:

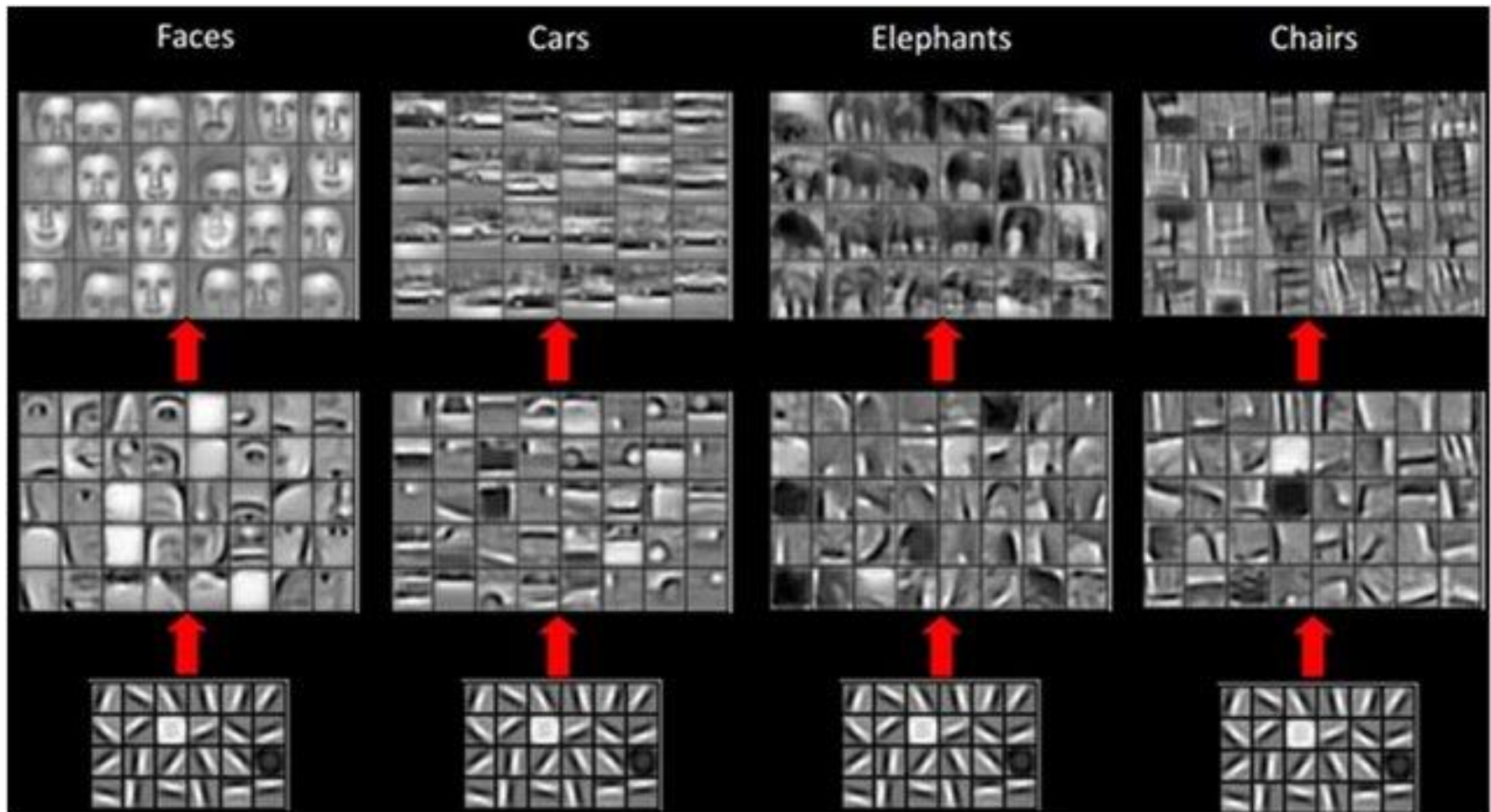
280

Problem: Consider an input of size $32 \times 32 \times 3$. Calculate the total number of trainable parameters for: 1. A fully connected layer with 20 output neurons. 2. A convolutional layer with 20 filters, each of size 5×5 .

Deep CNN Model



Feature Maps in Deep CNN Model



Loss Functions

- A **loss function** in deep learning is a mathematical function that quantifies the error between the model's predicted output and the actual target value.

$$\text{Loss} = f(y_{\text{true}}, y_{\text{predicted}})$$

where:

- y_{true} is the ground truth label.
- $y_{\text{predicted}}$ is the model's output.
- $f(\cdot)$ is the chosen loss function.

Types of Loss Functions

- Loss functions are generally categorized into two types:
- **A. Regression Loss Functions (For Continuous Outputs)**
- These loss functions are used when the model predicts a continuous numerical value.

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Penalizes larger errors more than smaller ones due to squaring.
- Suitable for cases where large deviations need more correction.

B. Classification Loss Functions (For Discrete Labels)

Used for problems where the model predicts class probabilities.

1. Binary Cross-Entropy (For Binary Classification)

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Used when the output is between 0 and 1 (sigmoid activation).
- Minimizing BCE encourages the model to push probabilities closer to the correct class.

2. Categorical Cross-Entropy (For Multi-Class Classification)

$$\text{CCE} = -\sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij})$$

- Used when multiple classes exist (softmax activation).
- Encourages the probability of the correct class to be maximized.

Optimizer

- An **optimizer** in deep learning is an algorithm that adjusts the model's parameters (weights and biases) to minimize the loss function and improve accuracy.

When a neural network makes predictions, it computes the loss (error). The optimizer's job is to **reduce this loss** by updating the model's parameters θ iteratively.

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial \theta}$$

where:

- θ represents model parameters (weights & biases).
- η is the **learning rate**, controlling how much to adjust the weights.
- $\frac{\partial \text{Loss}}{\partial \theta}$ is the **gradient**, which tells us the direction to move in order to minimize the loss.

Types of Optimizers

Optimizers can be categorized into two types:

A. First-Order Gradient Descent Methods

These optimizers rely only on first-order derivatives (gradients) to update parameters.

1. Stochastic Gradient Descent (SGD)

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla_{\theta} \text{Loss}$$

- Updates parameters **after every training sample**.
- Faster updates but **high variance**, leading to an unstable path toward convergence.
- Can get stuck in local minima.

2. Mini-Batch Gradient Descent

- Instead of updating after **one sample (SGD)** or the **entire dataset (Batch GD)**, mini-batch GD updates after a small group of samples.
- Reduces variance and speeds up computation.

B. Adaptive Optimizers (Second-Order Methods)

These optimizers adapt the learning rate dynamically for each parameter.

1. Adagrad (Adaptive Gradient Algorithm)

$$\theta_{\text{new}} = \theta_{\text{old}} - \frac{\eta}{\sqrt{G} + \epsilon} \nabla_{\theta} \text{Loss}$$

- Adjusts the learning rate based on past gradients.
- Works well for sparse data but learning rate decreases over time, making training slow.

2. RMSprop (Root Mean Square Propagation)

$$\theta_{\text{new}} = \theta_{\text{old}} - \frac{\eta}{\sqrt{E[\nabla_{\theta}^2] + \epsilon}} \nabla_{\theta} \text{Loss}$$

- Solves Adagrad's problem by using an exponential moving average of squared gradients.
- Keeps a stable learning rate for faster convergence.

3. Adam (Adaptive Moment Estimation) – Most Common

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \text{Loss}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \text{Loss})^2$$

$$\theta_{\text{new}} = \theta_{\text{old}} - \frac{\eta}{\sqrt{v_t} + \epsilon} m_t$$

- Combines momentum and RMSprop for fast and stable convergence.
- Works well across most deep learning tasks.

Transfer Learning

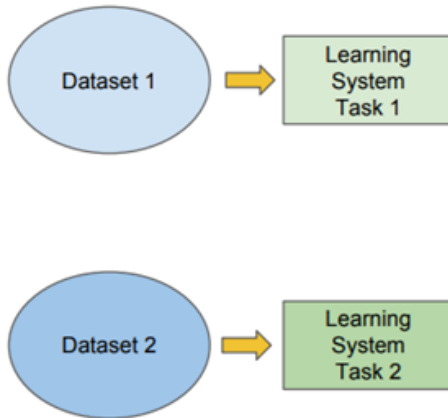
- Transfer learning is an approach to machine learning where a model trained on one task is used as the starting point for a model on a new task.
- This is done by transferring the knowledge that the first model has learned about the features of the data to the second model.
- In deep learning, transfer learning is often used to solve problems with limited data. This is because deep learning models typically require a large amount of data to train, which can be difficult or expensive to obtain.

Traditional ML

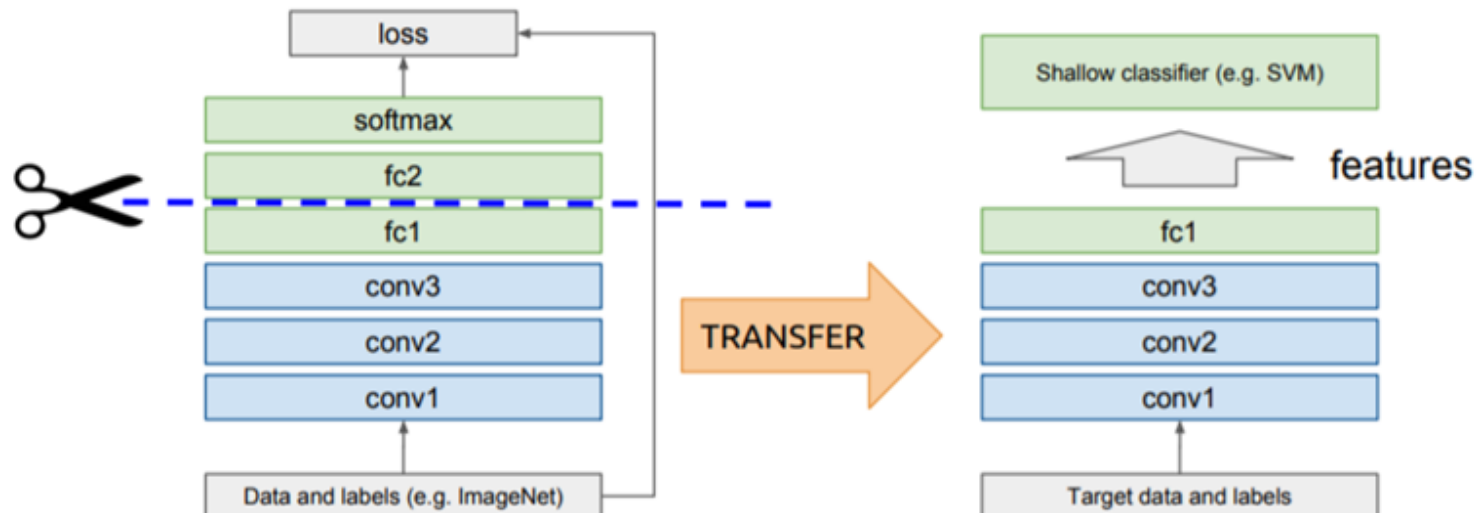
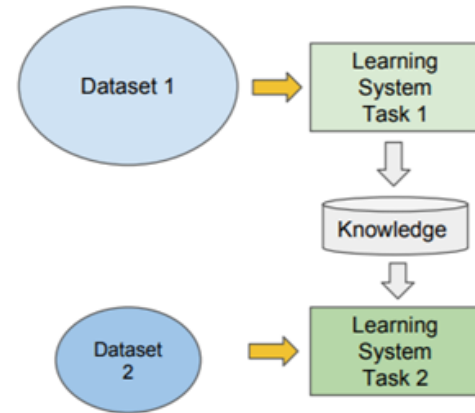
vs

Transfer Learning

- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



- Learning of a new tasks relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



Why Use Transfer Learning?

- **To save time and resources:** Training a deep learning model from scratch can be time-consuming and computationally expensive. Transfer learning can help you save time and resources by starting with a model that has already been trained on a large dataset.
- **To improve model performance:** Transfer learning can help you improve the performance of your model by transferring the knowledge that the pre-trained model has learned about the features of the data. This can be especially helpful if you have limited data for your target task.
- **To solve problems with limited data:** Transfer learning can be used to solve problems with limited data by transferring the knowledge that the pre-trained model has learned about the features of the data. This can be done by using feature extraction or fine-tuning.

- **Fine-tuning:**

- Fine-tuning is a machine learning technique in which a pre-trained model is further trained on a new dataset to improve its performance on a specific task.
- The pre-trained model is typically trained on a large dataset of general data, while the new dataset is specific to the task at hand.

- **What Is a Pre-Trained Model?**

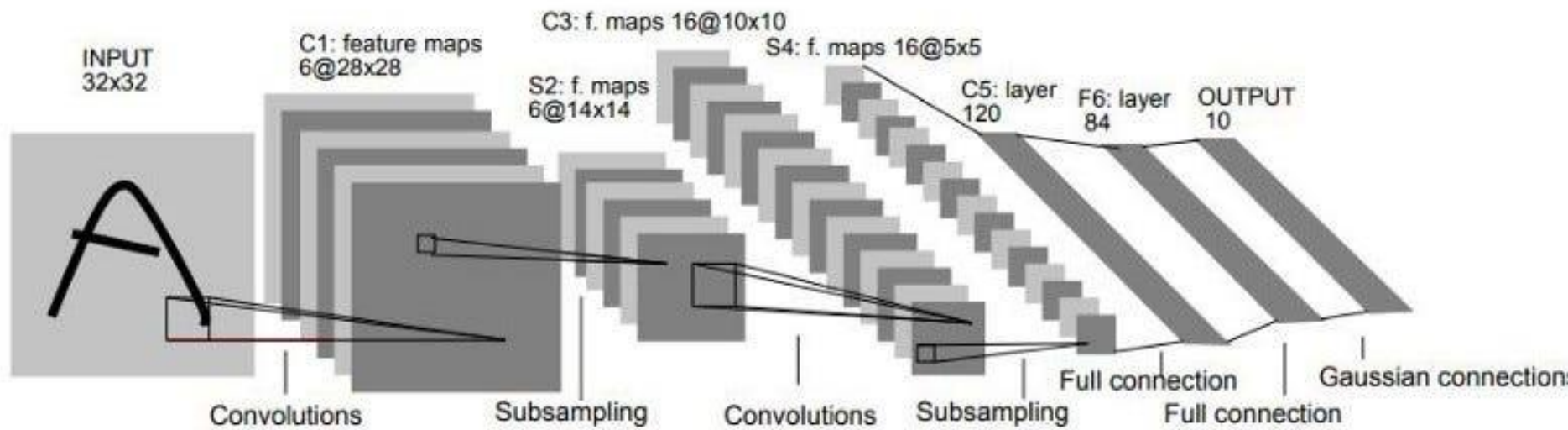
- A pre-trained model is trained on a large dataset. This dataset is typically much larger than the dataset that will be used to train the final model.
- The pre-trained model learns to extract features from the data, and these features can be used to train the final model more quickly and efficiently.

<i>Method</i>	<i>Feature Extractor</i>	<i>Classifier</i>
Transfer Learning	Pre-Trained (freeze)	Train From Scratch
Fine Tuning	Pre-Trained (freeze) Fine Tune	Train From Scratch

Difference between Transfer learning and Fine Tuning (by Elven Kim)

Popular Deep CNN Architectures

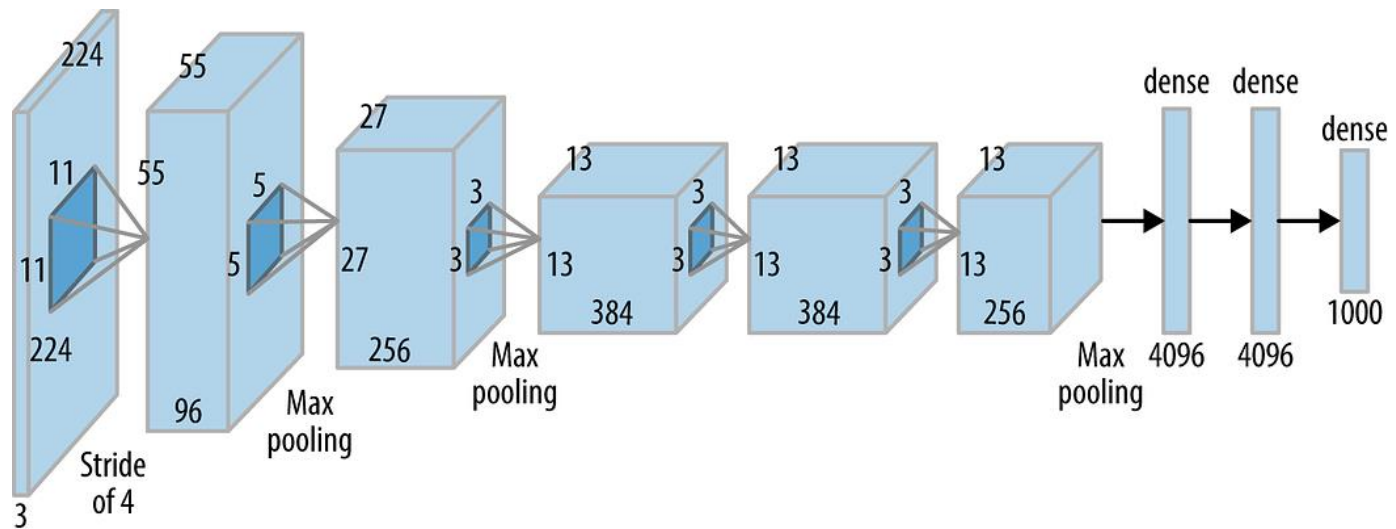
- **LeNet:** It is a convolutional neural network that Yann LeCun introduced in 1989.
- **Features of LeNet-5:**
 - Every convolutional layer includes three parts: convolution, pooling, and nonlinear activation functions.
 - 'tanh' is used as the activation function.
 - Using Multi-Layered Perceptron or Fully Connected Layers as the last classifier.
- **Architecture**
 - The LeNet-5 CNN architecture has seven layers.
 - Three convolutional layers, two pooling layers, and two fully connected layers.



	Layer	Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

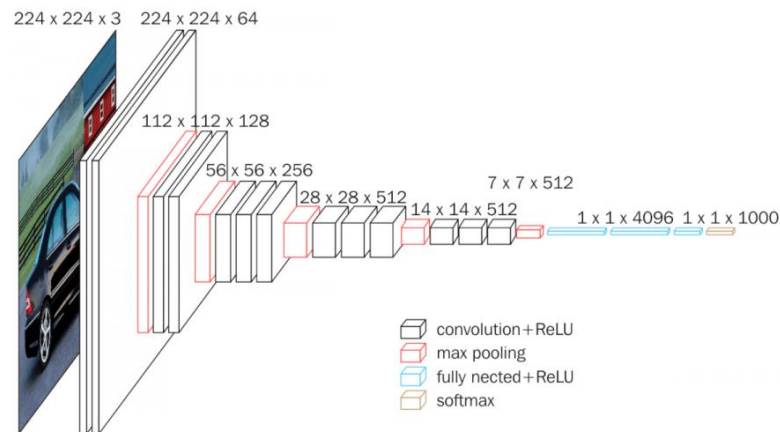
AlexNet

- The network consists of 5 Convolutional (CONV) layers and 3 Fully Connected (FC) layers. The activation used is the Rectified Linear Unit (ReLU).
- AlexNet receives an input of size $227 \times 227 \times 3$ and returns an output in the form of a 1000×1 probability vector, each element corresponding to a specific class.
- The entire network is composed of approximately 62 million trainable parameters.



• VGGNet:

- There exist multiple versions of VGGNet such as VGG16, VGG19, etc., differentiated by the total number of layers in the network.
- A critical aspect to highlight is that **all convolutional kernels are of size 3x3**, and maxpool kernels are 2x2 with a stride of two.
- VGG16 boasts around 138 million parameters.
- The real innovation lies in the 3x3 convolutions. Let's take an example. If we have an input layer of size 5x5x1, using a convolutional layer with a kernel size of 5x5 and stride one will result in an output feature map of 1x1.
- However, the same output feature map can be obtained by implementing two 3x3 convolutional layers with a stride of one.
- Hence, the 5x5 convolutional layer filter requires training 25 variables. In contrast, two convolutional layers of kernel size 3x3 require a total of $3 \times 3 \times 2 = 18$ variables, reducing the parameters by 28%.



VGG16 Block Diagram

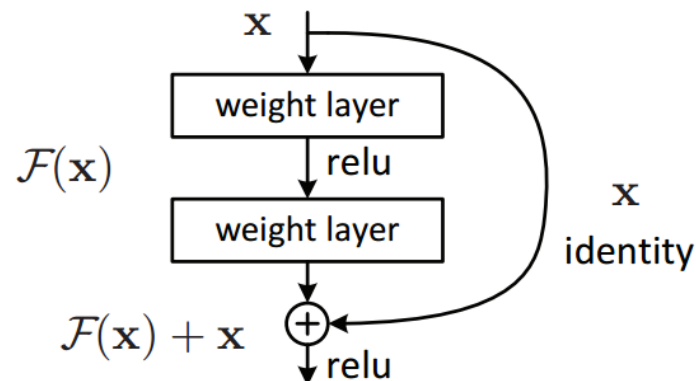
- **ResNet:**

- **The Problem:**

- As convolutional neural networks become deeper, the derivatives, when back-propagating to the initial layers, become almost negligible called vanishing gradient problem.

- **The Solution:**

- ResNet offers a clever remedy by introducing two types of 'shortcut connections': the *Identity shortcut* and the *Projection shortcut*.
- At the heart of ResNet lies the Residual block. Instead of learning the direct mapping from $x \rightarrow F(x)$, the network learns the residual mapping from $x \rightarrow F(x) + G(x)$.



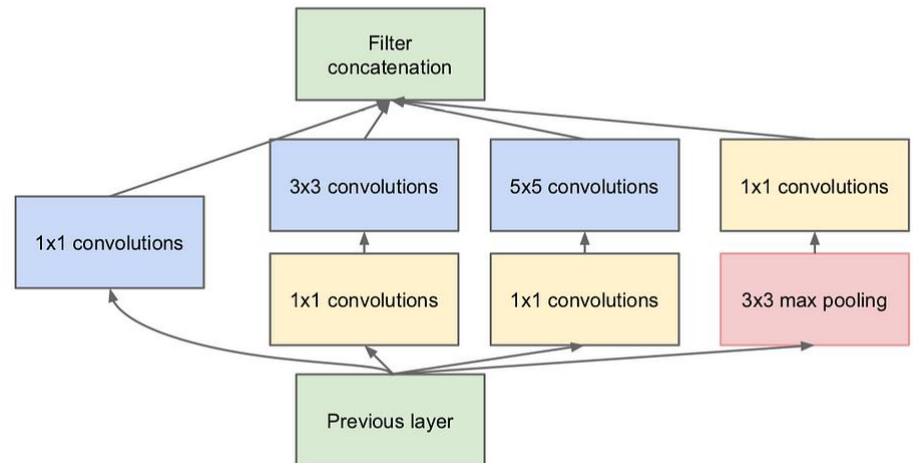
- ResNet boasts of several architectures with varying layer counts, such as ResNet50 and ResNet101.



<https://cv-tricks.com/keras/understand-implement-resnets/>

• InceptionNet (GoogleNet):

- Inception was motivated by the need to effectively recognize variable-sized features in image classification tasks.
- Instead of merely adding more layers (going deeper), Inception goes wider by incorporating kernels of different sizes in the same layer.
- The Inception network consists of several Inception modules, each comprising four parallel operations:
 - 1x1 conv layer
 - 3x3 conv layer
 - 5x5 conv layer
 - max pooling



• The Strategy:

- Inception expands the network space, letting training decide the most valuable features and weight accordingly. By allowing different convolutions to capture various features at a given level, all of these are extracted and concatenated before being fed to the next layer.

CNNs: A Comparative Snapshot

Comparison					
Network	Year	Salient Feature	top5 accuracy	Parameters	FLOP
AlexNet	2012	Deeper	84.70%	62M	1.5B
VGGNet	2014	Fixed-size kernels	92.30%	138M	19.6B
Inception	2014	Wider - Parallel kernels	93.30%	6.4M	2B
ResNet-152	2015	Shortcut connections	95.51%	60.3M	11B

<https://medium.com/@karandeepdps/alexnet-vggnet-resnet-and-inception-11880a1ed3cd>

Applications of CNN based Models

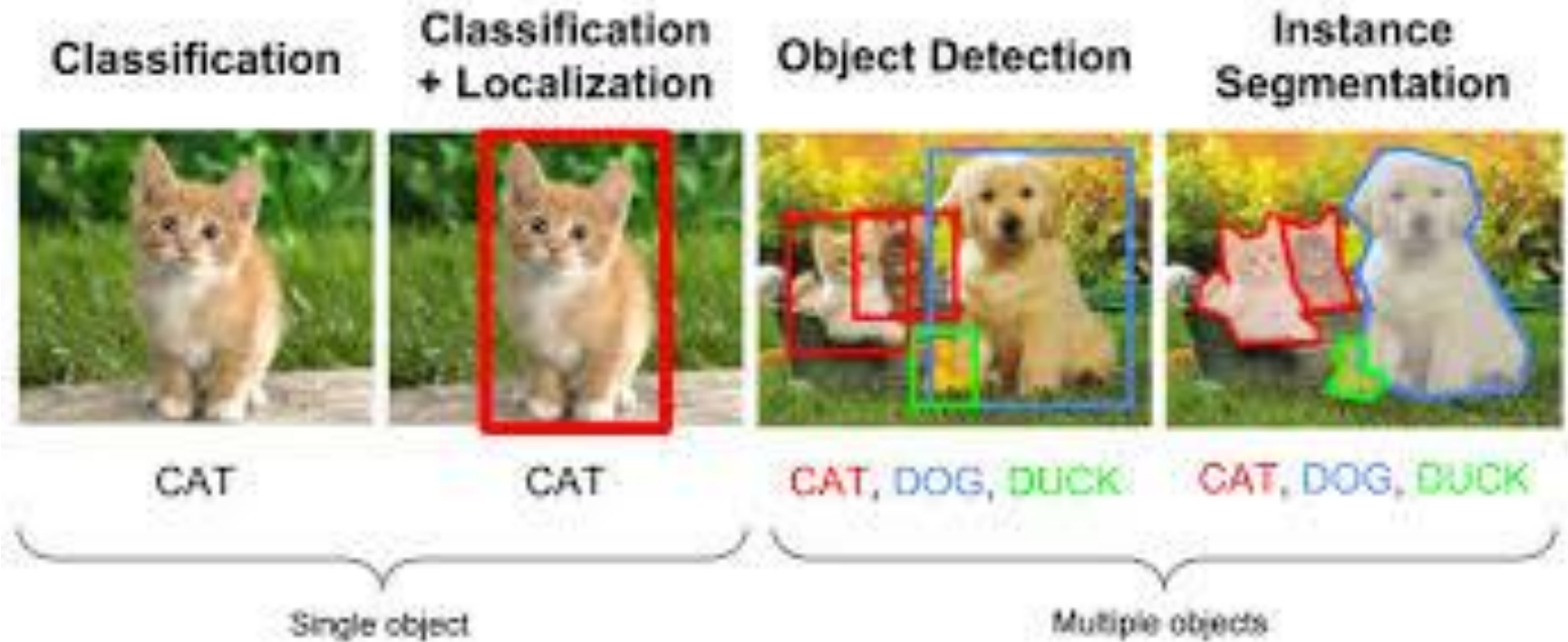
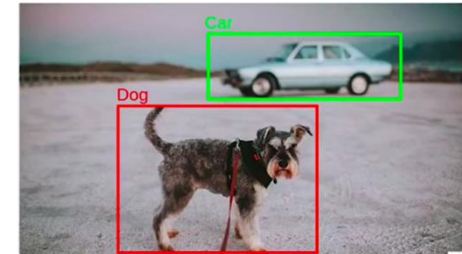


Image Classification

- Convolutional Neural Networks (CNNs) are widely used for image classification due to their ability to automatically learn features directly from image data.
- **Key Concepts:**
- **Feature Extraction with Convolutions:** CNNs use convolutional layers to extract local patterns like edges, textures, and shapes from images. These features are learned hierarchically: earlier layers detect simple edges, while deeper layers recognize complex shapes and objects.
- **Spatial Hierarchies:** Pooling layers reduce the spatial dimensions of the feature maps, preserving essential information while making the model computationally efficient. This helps CNNs to detect patterns regardless of their position within the image.
- **Fully Connected Layers for Classification:** After feature extraction, CNNs use fully connected layers to combine these features and classify the image into predefined categories. For example, in a CNN trained to identify animals, it may label an image as "cat" or "dog."
- **Popular Models:** AlexNet, VGGNet, ResNet, InceptionNet.
- **Applications:** CNNs are used in many image classification tasks, such as medical imaging (diagnosing diseases from X-rays), facial recognition, object detection, and more. Their success stems from their ability to generalize across variations in images, making them effective for real-world applications.
- **Advantages:** CNNs outperform traditional methods by learning features automatically, reducing the need for manual feature engineering, and providing high accuracy in complex image classification tasks.

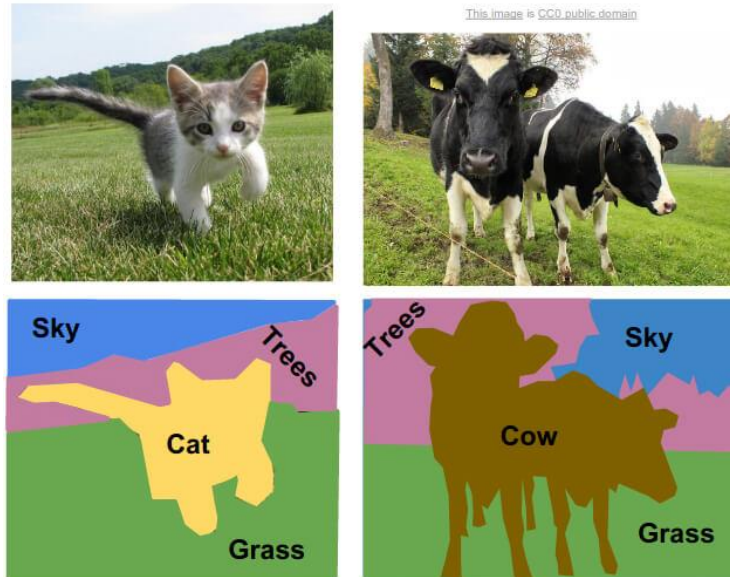
Object Detection

- Convolutional Neural Networks (CNNs) are fundamental in object detection, where the task is to locate and classify multiple objects within an image. Here's how CNNs are applied:
- **Key Concepts:**
- **Combining Classification and Localization:** Unlike image classification, object detection requires both the classification of objects and the localization of their positions within the image.
- **Region Proposal Networks (RPN):** Many CNN-based object detection models, like Faster R-CNN, use RPNs to generate potential regions or "proposals" that likely contain objects. These regions are refined and classified, speeding up the detection process.
- **Popular Architectures:**
 - R-CNN
 - YOLO (You Only Look Once)
 - SSD (Single Shot MultiBox Detector)
- **Applications:** CNN-based object detection is used in self-driving cars (to detect pedestrians and obstacles), surveillance (identifying suspicious activity), healthcare (detecting lesions in medical images), and more.
- **Advantages:** CNNs enable accurate, real-time object detection, handling diverse objects in complex environments. Their multi-scale feature extraction allows detection of both large and small objects within a single image.



Semantic Segmentation

- Convolutional Neural Networks (CNNs) are extensively used in semantic segmentation, where the goal is to classify each pixel in an image.
- For instance, in an image shown below, pixels belonging to a cat are labeled as "cat," pixels of a sky are labeled "sky" and so on. It provides a detailed understanding of the scene, as each pixel has its own label.



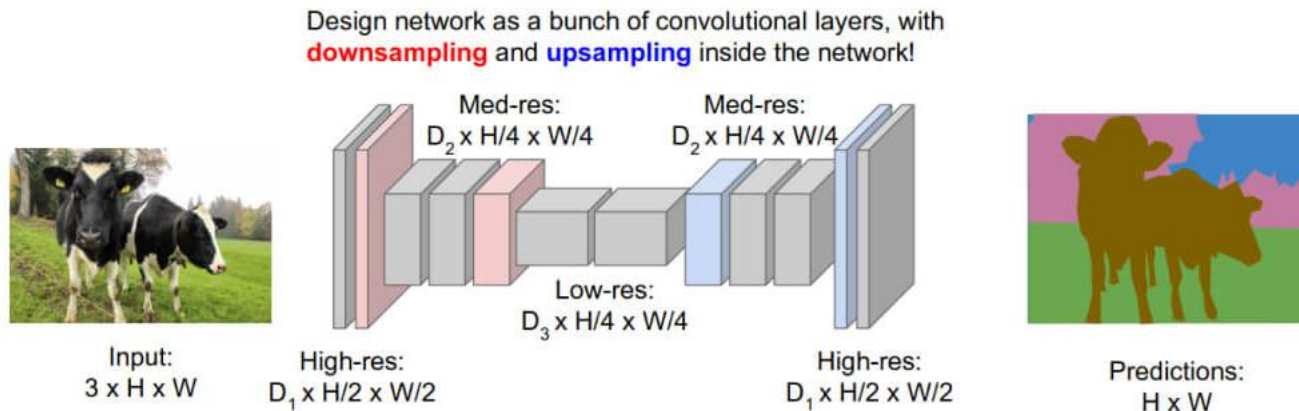
- **Why it's useful:** This is crucial in various applications like self-driving cars, where knowing exactly where each object is helps the car "see" and navigate safely.

Semantic Segmentation

- **Key Concepts:**
- **Pixel-Level Classification:** Unlike traditional image classification, semantic segmentation requires CNNs to label each pixel, distinguishing different objects and regions within the image, such as separating roads, trees, and buildings in an aerial image.
- **Encoder-Decoder Structure:** Many segmentation models, like U-Net, use an encoder-decoder architecture. The encoder extracts features and reduces the image size, while the decoder upsamples these features to the original image dimensions, creating a pixel-wise prediction map.
- **Skip Connections:** Networks like U-Net and Fully Convolutional Networks (FCNs) use skip connections to combine features from early layers (high-resolution details) with the upsampled layers in the decoder, preserving fine details crucial for accurate segmentation.
- **Popular Models:** FCN, U-Net.
- **Applications:** CNN-based semantic segmentation is used in autonomous driving (identifying roads, vehicles, pedestrians), medical imaging (segmenting organs or tumors), satellite image analysis, and environmental monitoring.
- **Advantages:** CNNs in semantic segmentation provide precise pixel-level predictions, allowing for a detailed understanding of complex scenes. Their feature extraction at multiple scales and use of skip connections enable accurate segmentation of both large and small objects within an image.

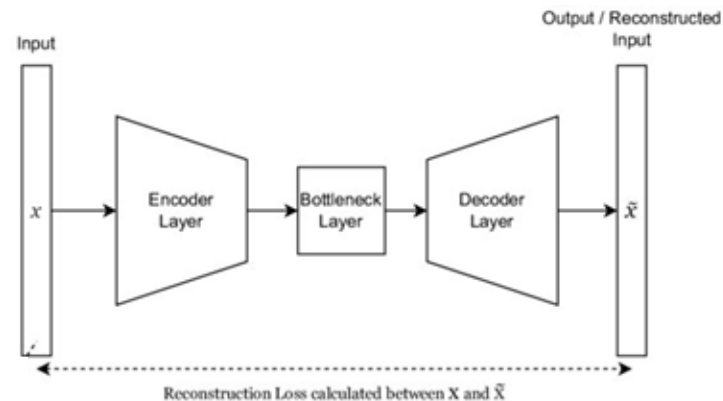
Fully Convolutional Network (FCN)

- The FCN was one of the first networks designed specifically for semantic segmentation.
- **Key Concepts of FCN:**
- **End-to-End Pixel Prediction:** FCNs replace fully connected layers with convolutional layers, allowing the network to take an image of any size and output a pixel-wise labeled map of the same spatial dimensions.
- **Upsampling:** After downsampling (reducing the image size through pooling layers), FCNs use upsampling to upsample the feature map back to the original input size.
- **Why it's important:** FCNs laid the groundwork for many other segmentation networks and showed that CNNs could perform pixel-level tasks!



Encoder-Decoder Architecture

- The Encoder-Decoder architecture is widely used in computer vision tasks such as image segmentation, denoising, and super-resolution. It helps transform an input image into a meaningful output image.
- The Encoder extracts key features from the image.
- The Decoder reconstructs the image into a required format.
- **Example:**
- **Input Image:** A satellite image of a city.
- **Output Image:** A segmented map highlighting roads, buildings, and vegetation.



How Encoder-Decoder Works for Images?

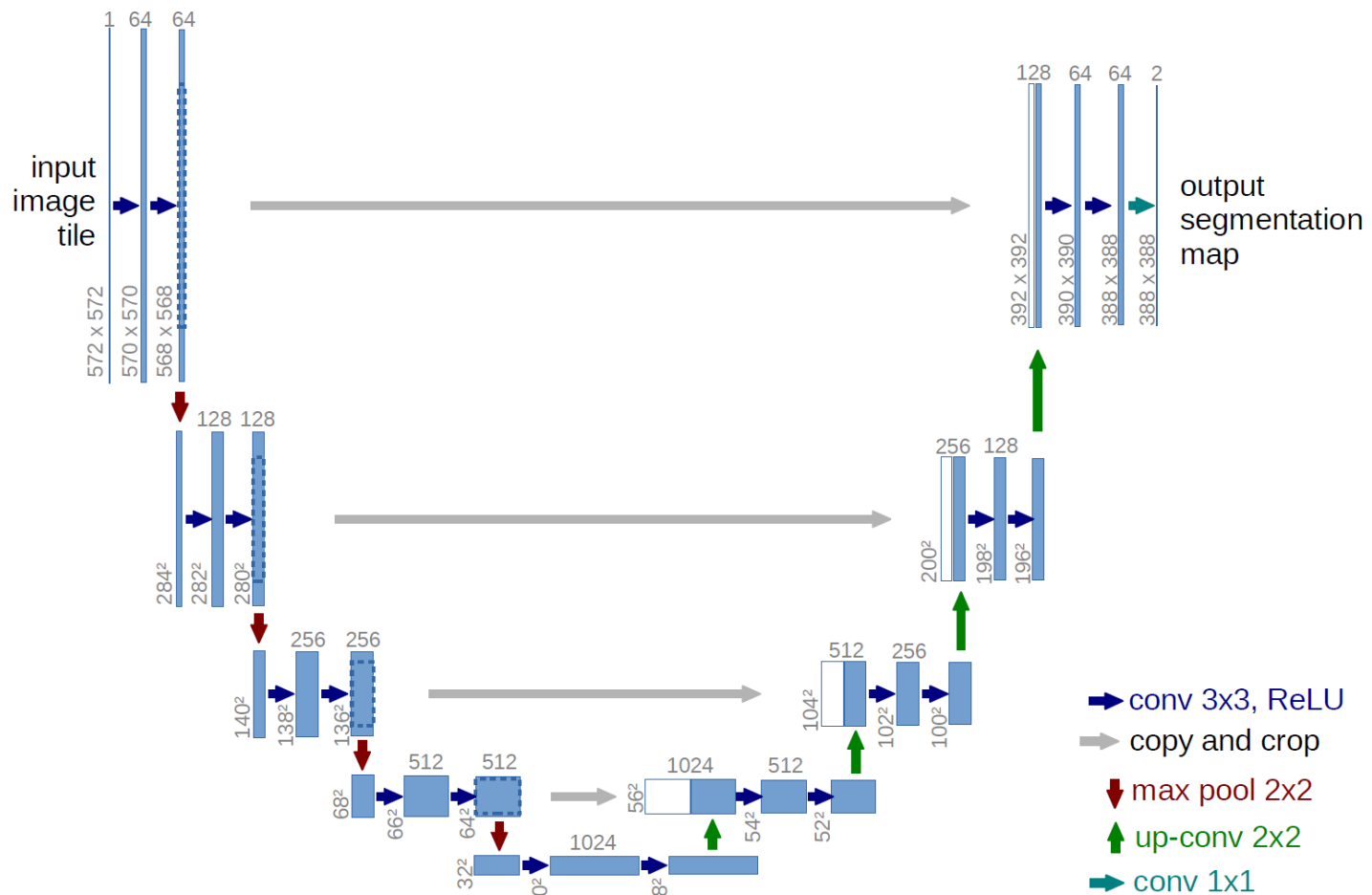
- **Encoder (Feature Extraction):**
 - The input image is passed through a series of convolutional layers (CNNs).
 - Each layer extracts key features (edges, textures, objects).
 - The image shrinks (downsampling) while retaining important features.
- **Bottleneck (Latent Representation):**
 - A compressed representation of the image is created.
 - This contains only essential information needed to reconstruct the output.
- **Decoder (Reconstruction):**
 - The decoder upsamples the compressed image.
 - It gradually reconstructs the output using transposed convolutions.
 - The final layer produces the desired image (segmented image, denoised image, etc.).

Applications of Image-Based Encoder-Decoder Model

- Semantic Segmentation (U-Net, SegNet) – Labeling each pixel of an image.
- Image Super-Resolution (SRCNN) – Enhancing low-resolution images.
- Image Denoising (Autoencoders) – Removing noise from old or blurry images.
- Style Transfer – Converting images to artistic styles.
- Image Colorization – Coloring black-and-white images.

U-Net

- U-Net is an extension of FCN, initially developed for biomedical image segmentation but widely used in various domains.
- **Key Concepts of U-Net:**
- **Encoder-Decoder Structure:** U-Net has a symmetric structure with an encoder (downsampling path) that captures features and a decoder (upsampling path) that reconstructs the image with pixel labels. This structure is often represented as a "U" shape.
- **Skip Connections:** U-Net also uses skip connections, connecting each layer in the encoder to the corresponding layer in the decoder. This allows it to preserve fine details lost during downsampling, which is crucial for accurate boundary detection.
- **Efficient for Small Datasets:** U-Net was designed to work well with limited labeled data, making it popular for applications like medical imaging where large datasets are often hard to obtain.
- **Why it's popular:** U-Net's structure is both simple and powerful, giving it the flexibility to be applied across many segmentation tasks while still maintaining accuracy even with limited data.



Architecture of U-Net

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18* (pp. 234-241). Springer International Publishing.

Autoencoder

- An Autoencoder is a special type of neural network designed to compress and reconstruct images. It learns to remove redundant information while keeping essential features.
- **Example:**
- **Input:** A noisy or blurry image.
- **Output:** A clean, denoised version of the same image.
- How Autoencoders Work for Images?

Application of Deep Learning in Medical Imaging

- **Why Medical Imaging?**

- Medical imaging techniques like X-ray, MRI, CT, and ultrasound help in diagnosing diseases.
- Traditional methods rely on human expertise, which can be time-consuming and error-prone.
- Deep Learning (DL) enhances accuracy and automation in medical diagnosis.

- **Why Deep Learning in Medical Imaging?**

- Learns features automatically (no manual feature extraction).
- Can detect patterns beyond human capability.
- Enables real-time and large-scale processing.

- **Applications in Medical Imaging:**
- **Disease Detection & Diagnosis**
 - **COVID-19 Detection** from X-ray images.
 - **Cancer Detection** (e.g., detecting lung and breast cancer in CT/MRI scans).
- **Segmentation & Localization**
 - **Tumor Segmentation** in MRI scans (e.g., brain tumor detection).
 - **Organ Segmentation** for surgical planning.
- **Classification & Risk Prediction**
 - Classifying different types of skin diseases from dermatology images.
 - Retinal disease detection from fundus images.
- **Enhancement & Restoration**
 - Super-resolution in MRI scans (improving image quality).
 - Noise reduction in low-quality X-rays.

- **Challenges & Ethical Considerations:**
- **Challenges:**
 - Data scarcity and quality.
 - Model interpretability (explainability issue).
 - Generalization to different populations.
- **Ethical Concerns:**
 - Privacy of patient data.
 - Bias in AI models (e.g., trained only on limited demographic data).

Applications of Deep Learning to NLP

- **What is NLP?**
- NLP stands for **Natural Language Processing**, which is a branch of AI that allows machines to understand, interpret, and generate **human language**.
- **Why Deep Learning for NLP?**
- Traditional ML models struggle with complex text data like sarcasm, context, and emotions.
- Deep learning models automatically learn features from text and capture complex patterns, making them more powerful for NLP tasks.
- **Popular Deep Learning Models for NLP:**
- RNN (Recurrent Neural Networks)
- LSTM (Long Short-Term Memory Networks)
- GRU (Gated Recurrent Unit)
- Transformers (like BERT, GPT)

Key Applications of Deep Learning in NLP

- **1. Sentiment Analysis:**

- **What it is:** Deep learning models can analyze text to determine the sentiment (positive, negative, or neutral) expressed.
- **How it works:** These models learn patterns in text data to identify sentiment-related keywords and phrases.
- **Examples:** Analyzing customer reviews, social media posts, or news articles to gauge public opinion.

- **2. Machine Translation:**

- **What it is:** Deep learning models translate text from one language to another with remarkable accuracy.
- **How it works:** These models learn the relationships between words and phrases in different languages, enabling them to generate fluent translations.
- **Examples:** Google Translate, Facebook's translation features, and other online translation services.

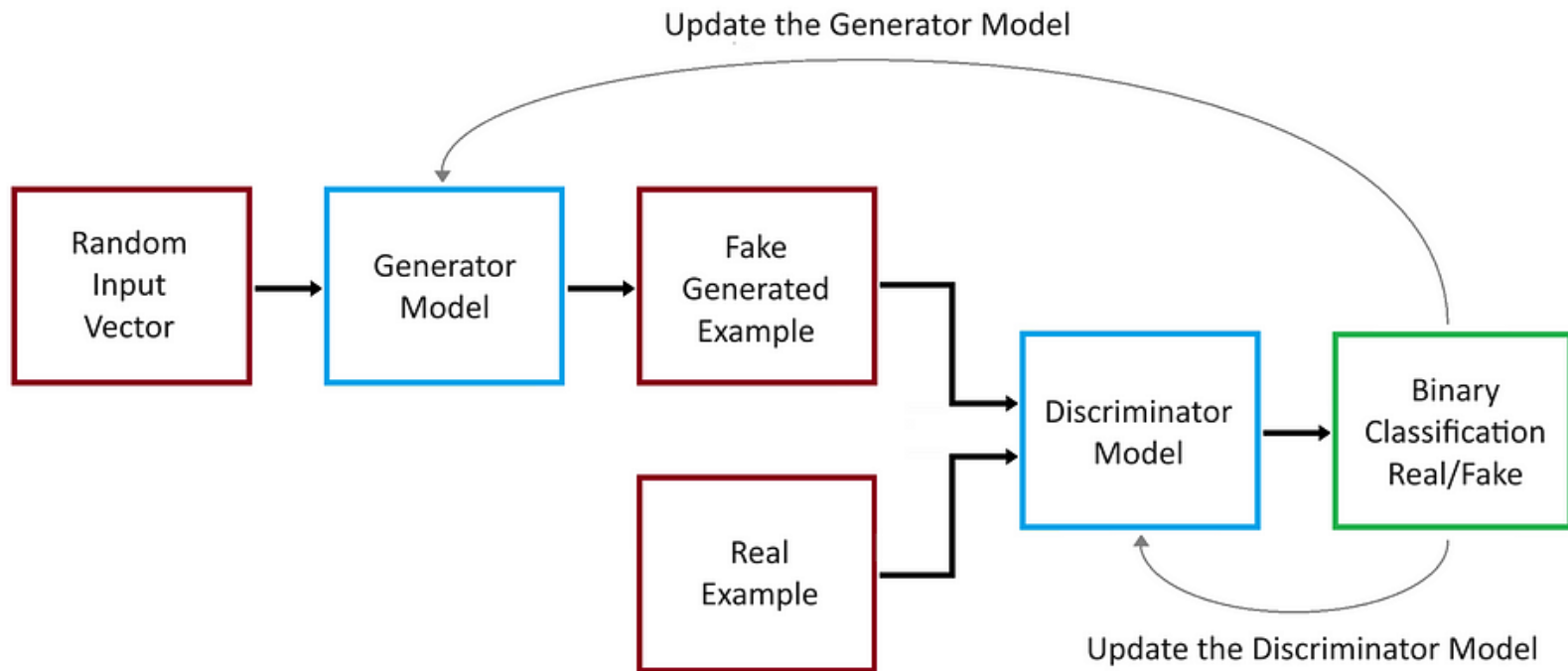
- **3. Text Generation:**
- **What it is:** Deep learning models can generate human-quality text for various purposes.
- **How it works:** These models learn from large text datasets to predict the next word or phrase in a sequence, allowing them to create stories, articles, or even code.
- **Examples:** Chatbots, content creation tools, and automated report generation.
- **4. Information Extraction:**
- **What it is:** Deep learning models can extract specific pieces of information from text, such as names, dates, locations, or organizations.
- **How it works:** These models learn to identify and categorize entities in text, allowing them to extract relevant data.
- **Examples:** Extracting customer addresses from emails, identifying products from online product descriptions, or finding keywords from news articles.

- **5. Text Summarization:**
- **What it is:** Deep learning models can generate concise summaries of long texts.
- **How it works:** These models learn to identify the most important sentences or phrases in a text, allowing them to create accurate and informative summaries.
- **Examples:** Summarizing news articles, research papers, or lengthy documents.

Application	Real-life Example	Deep Learning Model Used
Sentiment Analysis	Analyzing customer reviews on Amazon	LSTM, BERT
Machine Translation	Google Translate	Sequence-to-Sequence
Chatbots & Virtual Assistants	Siri, Alexa, ChatGPT	RNN, GPT-4
Text Summarization	Automatic news summarization	Transformer
Speech to Text	YouTube auto captions	Deep Speech Model
Fake News Detection	Identifying fake news on	BERT, RoBERTa

Generative Adversarial Network (GAN)

- Introduced by by Ian Goodfellow and his team in a 2014 NeurIPS paper.
- A GAN consists of two neural networks, namely the Generator and the Discriminator, which are trained simultaneously through adversarial training.
- **Generator:** This network takes random noise as input and produces data (like images). Its goal is to generate data that's as close as possible to real data.
- **Discriminator:** This network takes real data and the data generated by the Generator as input and attempts to distinguish between the two. It outputs the probability that the given data is real.

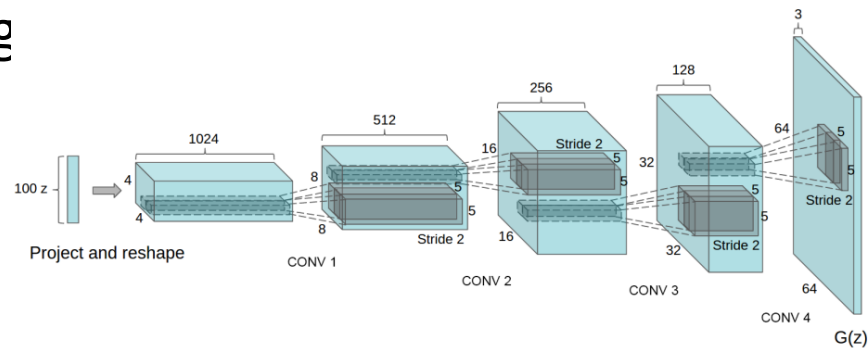


- During training, the Generator tries to produce data that the Discriminator can't distinguish from real data, while the Discriminator tries to get better at differentiating real data from fake data. The two networks are in essence competing in a game.
- This adversarial process leads to the Generator creating increasingly better data over time.

- **Generator:**

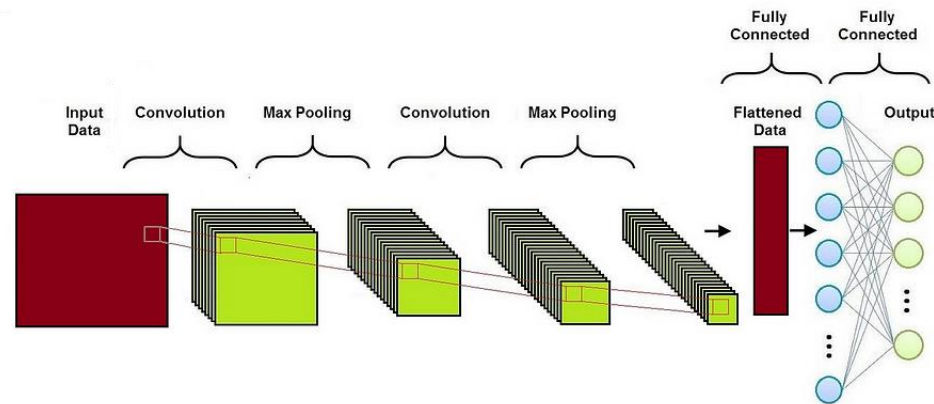
- **Input:** The generator takes in a random noise vector, usually sampled from a normal or uniform distribution. This vector serves as a seed or starting point for data generation.

- **Architecture:** Generators in many popular GANs (like DCGAN) are built using transposed convolutional layers (often called “deconvolutional” layers), which help in upsampling image.



- **Output:** The generator produces data as its output. In the case of a GAN designed for image generation, the output would be an image.

- **Discriminator:**
- **Input:** Receives real data and generated data.
- **Architecture:** The architecture of the discriminator is traditional deep convolutional neural networks (CNNs), but with some adjustments.



- **Output:** Provides a scalar value between 0 and 1, representing the probability that the input sample is real. A value close to 1 suggests the sample is likely real, while a value close to 0 suggests it's likely fake.

- **Model Training:**

- The **Discriminator** wants to:

- Output **1** for real data.
- Output **0** for fake data.

- So, the loss for the Discriminator is: $L_D = -[\log D(\text{real}) + \log(1 - D(\text{fake}))]$
- For real data, we want $D(\text{real}) = 1$, so $\log(D(\text{real})) \rightarrow$ maximum (which is 0 loss).
- For fake data, we want $D(\text{fake}) = 0$, so $\log(1 - D(\text{fake})) \rightarrow$ maximum (which is 0 loss).
- i.e., Discriminator **punishes itself** if it classifies real data as fake or fake data as real.

- The **Generator** tries to **fool the Discriminator**.

- It wants the **fake data to be c** $L_G = -\log D(\text{fake})$.

- So, the loss for the Generator is:

- The Generator wants $D(\text{fake}) = 1$, so that the fake data looks real.
- Hence, the Generator minimizes the loss when $D(\text{fake})$ is high.

- Discriminator wants to maximize the difference between real and fake.
- Generator wants to minimize the loss and make fake look real.

- Hence, **Overall GAN Loss:** $\min_G \max_D V(D, G)$

- **Applications:**

- **Image Generation:** This was one of the initial applications of GANs. They can be trained to generate high-resolution, realistic images from random noise.
- **Data Augmentation:** In domains where data is limited, GANs can be used to augment the dataset by generating new samples, which can be particularly valuable for training more robust machine learning models.
- **Style Transfer:** GANs can modify the style of images, such as converting photos into the style of famous paintings or changing day scenes to night scenes.
- **Super-Resolution:** Super-resolution GANs (SRGANs) can enhance the resolution of images, turning low-res images into high-res counterparts.
- **Image-to-Image Translation:** GANs can be used to translate images from one domain to another, such as turning satellite images into maps or sketches into colored images.
- **Generating Art:** Artists and hobbyists have used GANs to create original pieces of art, both in the form of static images and videos.

<https://machinelearningmastery.com/impressive-applications-of-generative-adversarial-networks/>